# Priority Scheduling (Non-Preemptive) এর মূল নিয়ম

- **প্রধান শর্ত:** মেমরিতে উপস্থিত (Arrived) প্রসেসগুলোর মধ্যে যার প্রায়োরিটি সবচেয়ে বেশি (এখানে ছোট সংখ্যা = বেশি প্রায়োরিটি), সেটি আগে এক্সিকিউট হবে।
- **Tie-breaker:** যদি দুই বা ততোধিক প্রসেসর প্রায়োরিটি একই হয়, তবে যার **Arrival Time (AT)** কম তাকে আগে সুযোগ দেওয়া হবে।
- **Non-Preemptive:** একবার একটি প্রসেস শুরু হলে সেটি শেষ না হওয়া পর্যন্ত CPU অন্য কোনো প্রসেস ধরবে না।

---

## গাণিতিক সূত্রসমূহ (SJF এর অনুরূপ)

Priority Scheduling-এর ক্ষেত্রেও হিসাবগুলো একই থাকে:

- **Completion Time (CT):** প্রসেসটি যখন শেষ হয়। ct[idx]=time;
- **Turnaround Time (TAT):** TAT = CT - AT
- **Waiting Time (WT):** WT = TAT - BT
- **Response Time (RT):** Non-preemptive এর ক্ষেত্রে RT = WT।

---

## QUES: Priority Scheduling

| Process | BT | AT | PR | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| **P1** | 5 | 3 | 2 | | | | |
| **P2** | 4 | 2 | 1 | | | | |
| **P3** | 1 | 5 | 3 | | | | |
| **P4** | 7 | 4 | 4 | | | | |
| **P5** | 6 | 1 | 3 | | | | |
| **P6** | 2 | 0 | 5 | | | | |

Solve table:

| Process | BT | AT | PR | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| **P1** | 5 | 3 | 2 | 11 | 8 | 3 | 3 |
| **P2** | 4 | 2 | 1 | 6 | 4 | 0 | 0 |
| **P3** | 1 | 5 | 3 | 18 | 13 | 12 | 12 |
| **P4** | 7 | 4 | 4 | 25 | 21 | 14 | 14 |
| **P5** | 6 | 1 | 3 | 17 | 16 | 10 | 10 |
| **P6** | 2 | 0 | 5 | 2 | 2 | 0 | 0 |

**Gantt Chart:**

| P6 | P2 | P1 | P5 | P3 | P4 |
|---|---|---|---|---|---|

0 2 6 11 17 18 25

# Code:

```
#include <stdio.h>
int main()
{
    int n,i,cnt=0,time=0,idx;
    int pid[20],bt[20],at[20],pr[20],wt[20],tat[20],ct[20],vis[20]={0};

    printf("Enter the number of processes: ");
    scanf("%d",&n);

    printf("Enter process IDs:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pid[i]);

    printf("Enter burst times:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    printf("Enter arrival times:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter priorities (lower number = higher priority):\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    while(cnt<n)
    {
```

```c
        idx=-1;
        for(i=0;i<n;i++)
        {
            if(!vis[i] && at[i]<=time)
            {
                if(idx == -1 || pr[i] < pr[idx] || (pr[i] == pr[idx] &&
at[i] < at[idx]))
            //  if(idx == -1 || pr[i] > pr[idx] || (pr[i] == pr[idx] &&
at[i] < at[idx])) higher number = higher priority
                {
                    idx=i;
                }

            }
        }

        if(idx==-1)
        {
            time++;
            continue;
        }

        time+=bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];

        vis[idx]=1;
        cnt++;
    }

    float total_wt = 0, total_tat = 0;

    printf("\nPID\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",pid[i], at[i], bt[i], pr[i],
ct[i], wt[i], tat[i]);

        total_wt += wt[i];
        total_tat += tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

    return 0;
}
```

# বিশেষ দ্রষ্টব্য (PDF অনুযায়ী পরিবর্তন)

**সতর্কতা:** যদি প্রশ্নে বলা থাকে Higher number = Higher Priority", তবে কোডের কন্ডিশন `pr[i] < pr[idx]` এর পরিবর্তে `pr[i] > pr[idx]` ব্যবহার করতে হবে (যা আপনার কোডে কমেন্ট হিসেবে দেওয়া আছে)।

Lower number = Higher Priority

```
if(idx == -1 || pr[i] < pr[idx] || (pr[i] == pr[idx] && at[i] < at[idx]))
```

**Higher number = Higher Priority**

```
if(idx == -1 || pr[i] > pr[idx] || (pr[i] == pr[idx] && at[i] < at[idx]))
```